

Neural Network Training Revisited

Applied Deep Learning — Chapter 16

Yin Yang

Foundation Books Project

Roadmap

Conceptual Roadmap and Running Example

What Makes Training Good?

Logits, Soft Targets, and Temperature

Knowledge Distillation

Distilling Transformer Classifiers

Fast and Stable Training Tools

Compression Beyond Distillation

Data, Regularization, Calibration

Experiment Design and Debugging

Case Study: Distilling a Small Transformer

Summary

Conceptual Roadmap and Running Example

From Fitting to Applied Constraints

After attention, LLMs, LoRA, and RL post-training: training is no longer only “fit a model from data.”

- The strongest model is often already large, pretrained, and expensive.
- Question becomes: is a smaller or cheaper model accurate enough, fast enough, and reliable enough?
- A training run is **evidence**, not just a completed command.
- Sequence length, tokenizer, split, and timing device all change what was measured.

Running Example: Transformer News Classifier

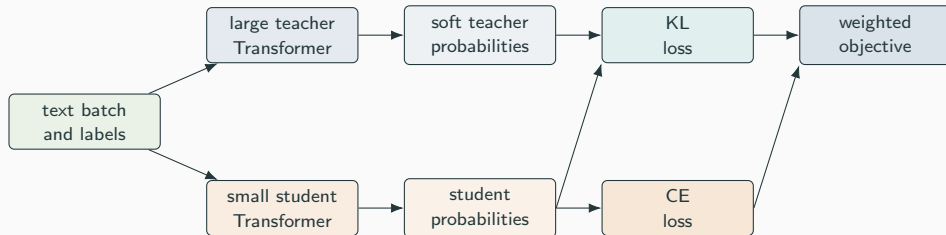
Task: assign articles to *world*, *sports*, *business*, or *science and technology* (e.g. AG News).

Teacher larger pretrained Transformer classifier — accurate but expensive.

Student smaller Transformer classifier — faster or smaller, but weaker on its own.

Useful contrast: expensive expert vs. fast apprentice. The metaphor earns its place only when the report measures the tradeoff.

Teacher-Student Distillation Workflow



Teacher fixed; only student parameters move. Student learns from *labels and* teacher probabilities.

Applied constraint

A measurable requirement: max latency, max model size, min validation accuracy, memory limit, training-time budget, hardware target.

Teacher: 94% accuracy, 40 ms/example. App needs answer in ≤ 10 ms. $\Rightarrow$ teacher not deployable.

Key baseline

A student with the *same architecture* trained only from hard labels.

Comparison must include teacher, hard-label student, and distilled student.

What Makes Training Good?

Accuracy Is Necessary But Incomplete

A useful model must be accurate *and*

- fit the available hardware,
- answer fast enough,
- be reproducible.

Aggregate accuracy hides class structure: “mostly correct” chip-earnings articles labeled as science/tech might be unacceptable for a finance app. **Show class-level errors, confidence, and teacher–student disagreement before claiming the smaller model is ready.**

Latency elapsed time for one example or one specified batch.

Throughput examples per unit time under a specified setup.

Parameter count learned scalar values.

Model size on disk bytes of saved artifact.

Teacher: 40 ms; student: 8 ms $\Rightarrow$ measured speedup $5\times$. State hardware, batch size, max sequence length, precision, warmup, number of repetitions. Fewer parameters $\neq$ faster.

Split Discipline and the Minimum Report

Distillation makes split discipline easier to violate.

- Teacher, hard-label student, distilled student must share the split.
- Validation chooses temperature, α , learning rate, checkpoint.
- Test set answers only the **final selected-model** question.

Minimum applied report: one row per model with validation accuracy, test accuracy, latency, throughput, parameter count, model size, training time.

Logits, Soft Targets, and Temperature

Hard Targets vs. Soft Targets

Four-class problem, correct class is *business*.

- Hard target: $(0, 0, 1, 0)$.
- Teacher might output: $(0.04, 0.01, 0.80, 0.15)$.

Soft target says: business is most likely; science/tech is plausible second; sports and world are unlikely. *Hard labels hide this.*

Temperature-Scaled Softmax

$$p_k^{(T)} = \frac{\exp(z_k/T)}{\sum_{j=1}^K \exp(z_j/T)}.$$

- $T > 1$: softer, less peaked distribution.
- Ranking among classes is preserved.
- Useful in distillation because it exposes how the teacher compares *wrong* answers.

Temperature is not a confidence-repair knob. A confidently wrong teacher only becomes more visibly wrong.

Cross-Entropy as Target Matching

$$\text{CE}(q, p) = - \sum_{k=1}^K q_k \log p_k.$$

Hard target on class 1, student assigns 0.665:

$$-\log(0.665) \approx 0.408.$$

Student assigns 0.10:

$$-\log(0.10) \approx 2.303.$$

Cross-entropy penalizes low probability on the target class. Distillation uses the same idea but the target is the teacher's full distribution.

Knowledge Distillation

Knowledge distillation

A training procedure where a student model learns from teacher outputs, intermediate teacher representations, or both — often in addition to ordinary labeled examples.

- Bucila–Caruana–Niculescu-Mizil (2006): model compression by mimicking larger models.
- Hinton–Vinyals–Dean (2015): temperature-based formulation.

Core applied question: **can the student keep enough teacher behavior while being cheaper to run?**

KL Divergence for Teacher Matching

$$\text{KL}(q \parallel p) = \sum_{k=1}^K q_k \log \frac{q_k}{p_k}.$$

Teacher $q = (0.70, 0.20, 0.10)$, student $p = (0.60, 0.30, 0.10)$:

$$0.70 \log \frac{0.70}{0.60} + 0.20 \log \frac{0.20}{0.30} + 0.10 \log \frac{0.10}{0.10} \approx 0.027.$$

Individual terms can be negative; total is always nonnegative.

The Weighted Distillation Loss

$$\mathcal{L}_{\text{student}} = \alpha \text{CE}(y, p_s) + (1 - \alpha) T^2 \text{KL}\left(p_t^{(T)} \parallel p_s^{(T)}\right).$$

- $\alpha = 1$: ordinary hard-label training.
- $\alpha = 0$: ignores human labels.
- Both signals usually useful: label anchors task; teacher provides smoother target.
- T^2 keeps gradient scale comparable across temperatures.

When Distillation Earns Attention

Possible reasons it helps:

- Soft targets act like regularization.
- Teacher learned similarities the student couldn't infer alone.
- Teacher summarizes more evidence (ensemble, larger model).

Most important comparison: same student architecture, hard-label vs. distilled, identical split/budget/checkpoint rule. Hard-label 89%, distilled 91% $\Rightarrow$ method earned attention. Both 91% $\Rightarrow$ teacher gave nothing extra here.

Distilling Transformer Classifiers

Transformer Inputs Must Be Held Fixed

Tokenizer converts raw text to token IDs, padding, truncation.

Attention mask marks real input vs. padding positions.

Hold inputs fixed. If teacher reads 256 tokens and student reads 64, the comparison is no longer only about distillation. Broken attention masks produce strange “training” bugs that are really data-pipeline bugs.

Logit Distillation for Classifiers

Sequence-classification head: pooled representation $\rightarrow$ topic logits. Logit distillation:

1. Teacher and student read the same tokenized article.
2. Both produce class logits.
3. Loss encourages student probabilities to match teacher probabilities at temperature T .

Simplest, most direct beginner implementation. Other variants: *feature* matching, *attention-relation* matching (DistilBERT, TinyBERT, MiniLM).

Feature and Attention Distillation

Feature matching:

$$\mathcal{L}_{\text{feat}} = \sum_{\ell \in S} \left\| P_{\ell} H_{\text{student}}^{(\ell)} - H_{\text{teacher}}^{(m(\ell))} \right\|_2^2.$$

Attention matching:

$$\mathcal{L}_{\text{attn}} = \sum_{\ell \in S} \left\| A_{\text{student}}^{(\ell)} - A_{\text{teacher}}^{(m(\ell))} \right\|_2^2.$$

Powerful but introduces alignment problems: layer maps $m(\ell)$, projections P_{ℓ} , width mismatches. **Logit distillation first.**

A Controlled Transformer Recipe

1. Load or fine-tune a teacher; verify it beats a simple baseline.
2. Train the small student from hard labels only.
3. Train the same student with the distillation loss.
4. Compare three rows on the same validation set and timing protocol.

If the distilled student is not better, do not hide it: inspect T , α , learning rate, sequence length, teacher quality, class imbalance.

Fast and Stable Training Tools

Mixed Precision and Gradient Accumulation

Mixed precision FP16/BF16 ops for speed; preserve enough FP32 for stability (loss scaling for FP16).

Gradient accumulation gradients on several mini-batches, then one optimizer step.

Per-device batch 8, accumulation 4, single device $\Rightarrow$ effective batch 32. Report effective batch.

Per-device batch $\times$ accumulation $\times$ devices.

Gradient Checkpointing and Pipeline Bottlenecks

Gradient checkpointing (activation checkpointing)

Store only selected forward activations; recompute the rest during backprop.

- Lower peak memory, slower backward (recomputation cost).
- Helps when activations dominate (long sequences, deep stacks).
- Not a model checkpoint on disk — a memory strategy inside one training step.

Input pipeline can dominate: GPU waiting for tokenization or batching is a data-loading bug, not a model bug.

Stability and Checkpoint Selection

Gradient clipping limits gradient magnitude to reduce unstable updates. Useful diagnostic, not a fix for a wrong learning rate.

Checkpoint saved model state.

Early stopping stop based on validation behavior, not fixed epochs.

Checkpoint choice must be consistent across hard-label and distilled runs. Comparing “final epoch” against “best validation epoch” biases the result.

Compression Beyond Distillation

Pruning, Quantization, Low Rank

Pruning remove weights, neurons, channels, heads, or layers.

Quantization store / compute with lower-precision numbers.

Low rank replace $W \in \mathbb{R}^{m \times n}$ by AB , $A \in \mathbb{R}^{m \times r}$, $B \in \mathbb{R}^{r \times n}$, $r \ll \min(m, n)$.

Pruned weights: $W_{\text{pruned}} = M \odot W$, $M \in \{0, 1\}^{m \times n}$. **Smaller is not faster.** Unstructured sparsity often needs special kernels or hardware before latency improves.

Dense head: $768 \rightarrow 4$ classes, 3076 parameters.

- FP32: $3076 \times 4 = 12,304$ bytes of weights.
- One byte per value: ≈ 3076 bytes.

Deployment quantization step:

$$q = \text{round}\left(\frac{w - z}{s}\right), \quad \hat{w} = s q + z.$$

Post-training quantization converts after training; *quantization-aware training* simulates it during training.

Data, Regularization, Calibration

Class Imbalance and Label Smoothing

80% of examples are sports $\Rightarrow$ predict “sports” always $\Rightarrow$ 80% accuracy. Useless to a router needing rare classes. Pair accuracy with per-class metrics or a confusion matrix.

Label smoothing

Hard target $(1, 0, 0, 0)$ with $\epsilon = 0.1$ becomes $(0.9, 0.033, 0.033, 0.033)$.

Different from distillation (no teacher), but shares the idea that less extreme targets can help training.

Calibration

Predicted confidence agrees with empirical correctness.

100 predictions at confidence ≈ 0.8 , 65 correct $\Rightarrow$ overconfident: behaves like 65% but speaks like 80%.

$$\text{ECE} = \sum_{m=1}^M \frac{|B_m|}{n} |\text{acc}(B_m) - \text{conf}(B_m)|.$$

Confidence is a claim. ECE depends on binning; complement with error inspection.

Experiment Design and Debugging

Ablation change one component at a time and observe its effect.

Overfit test train on 32–64 examples and verify the loss drops.

Useful ablations: T , α , student size, max sequence length, learning rate. Overfit catches:
broken labels, frozen parameters, wrong loss signs, bad attention masks, optimizer mistakes.

Error Groups That Tell Different Stories

- Teacher right, student wrong $\Rightarrow$ student capacity limit.
- Hard-label right, distilled wrong $\Rightarrow$ harmful teacher bias copied.
- All models wrong $\Rightarrow$ ambiguous labels or missing context.

Teacher can be confidently wrong. Soft targets may train the student to reproduce that mistake more smoothly than hard labels would. Lowering $1 - \alpha$, filtering weak teacher examples, or keeping a hard-label baseline is more sensible than treating teacher confidence as ground truth.

Debugging Order When Distillation Fails

1. Verify the data split and labels.
2. Verify the teacher is strong on validation.
3. Verify the hard-label student trains.
4. Check distillation targets sum to one.
5. Check temperature applied consistently to teacher and student.
6. Check KL direction is teacher-to-student.

Only *after* these checks should you blame the method.

Case Study: Distilling a Small Transformer

AG News Case-Study Setup

- Four labels: world, sports, business, science/technology.
- 4000 train, 1000 val, 1000 test.
- Teacher: `textattack/bert-base-uncased-ag-news`.
- Student: `prajjwal1/bert-mini`.
- Shared: tokenizer, max seq length 96, batch 32, 3 epochs.
- Distilled student: $\alpha = 0.5$, $T = 2$.
- Hardware: CUDA, BF16 autocast, 8 warmup + 40 timed reps.

Tradeoff Table

Model	Val acc.	Test acc.	Latency	Size
Teacher (BERT-base)	99.6%	94.3%	5.84 ms	417.7 MB
Hard-label student	90.0%	87.7%	2.39 ms	42.6 MB
Distilled student	90.0%	88.1%	2.43 ms	42.6 MB

- Distilled student tied hard-label on validation, slightly higher on test (88.1 vs. 87.7).
- Both compact: $\approx 9.8\times$ smaller, $\approx 2.4\times$ lower latency vs. teacher.
- **Claim the size of the win.** Cautious language for small differences.

The Deployment Decision Is Conditional

If $\approx 88\%$ on this 1000-example test split is acceptable:

- Either compact student is a plausible candidate.
- Distilled has slightly higher test accuracy.
- Hard-label is slightly faster.
- Difference too small to declare a universal winner.

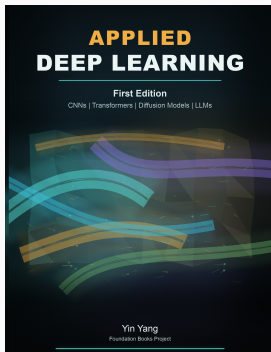
If application needs $\sim 94\%$: **this student is too small**. Try larger student, more data, or another distillation recipe. Goal is a defensible tradeoff, not automatic student victory.

Summary

Summary

- A strong training result is an **evidence package**: task, split, model, run, validation protocol, timing, parameters, size, memory, failures.
- Distillation = logits, soft targets, temperature, CE, KL, weighted objective.
- Compare distilled student against the same student trained from hard labels — not only against the teacher.
- Tools around the loss matter: mixed precision, gradient accumulation, checkpointing, clipping, label smoothing, calibration.
- Pruning, quantization, low-rank: smaller representation $\neq$ faster; latency must be measured on the target runtime.

**Next: Vision Transformers — patches, attention,
pretraining, and the same evidence discipline applied to
images.**



Applied Deep Learning

Chapter overview slides for the book by Yin Yang.

Book page	foundation-books.github.io/applied-deep-learning
Code	github.com/foundation-books/applied-deep-learning-code
Print	amazon.com/dp/B0GZF7QRSH
Kindle	amazon.com/dp/B0GZF9221X
License	CC BY-NC-ND 4.0

These free overview slides may be shared non-commercially with attribution and without modifications. Full explanations, exercises, and companion-code context are in the book and linked repository.